□ 서식4_한이음 드림업 프로젝트 결과보고서

2026년 한이음 드림업

프로젝트 결과보고서

프로젝트명	공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발

요 약 본

프로젝트 정보	
프로젝트명	공공데이터 AI 데이터셋 자동 표준화 및 검증 플랫폼 개발
주제 영역	☐ 생활 ☐ 업무 ■ 공공/교통 ☐ 금융/핀테크 ☐ 의료 ☐ 교육 ☐ 유통/쇼핑 ☐ 엔터테인먼트
기술 분야	■ SW·AI ☐ 방송·콘텐츠 ☐ 블록체인·융합 ☐ 디바이스 ☐ 차세대보안 ☐ 미래통신·전파
달성 성과	☐ 논문게재 및 포스터 발표 ☐ 앱등록 ☐ 프로그램등록 ☐ 특허 ☐ 기술이전 ☐ 실용화 ☐ 공모전( ) ☐ 기타( ) 해당 성과 체크
프로젝트 소개	• 공공데이터를 수집·정제·검증해 분석과 AI 학습에 바로 쓸 수 있는 테이블과 데이터셋으로 만드는 플랫폼 • 실행 엔진 KPubData Builder와 웹 작업공간 KPubData Studio로 구성 • 공공 API 접근은 자체 라이브러리 kpubdata가 담당
개발 배경 및 필요성	• 공공데이터는 개방되어 있지만 기관마다 형식이 달라 바로 분석하기 어려움 • 정제 규칙과 처리 이력이 흩어지면 같은 결과를 다시 만들거나 오류 원인을 찾기 힘듦 • 규칙을 명세로 선언하고 실행 결과를 기록하는 도구가 필요
프로젝트 특·장점	• 명세(BuildSpec) 하나로 수집부터 내보내기까지 선언하고 실행 전에 검증 • 실행마다 출처·행 수·품질 결과를 Manifest로 남기고 테이블은 스냅샷으로 관리 • 코드 없이 웹 화면에서 테이블 생성, SQL 분석, 리포트 작성 가능
주요 기능	• 소스 카탈로그 탐색과 3단계 테이블 만들기 • Bronze·Silver·Gold 단계 처리, 품질 검사, 개인정보 검사와 마스킹 • 스냅샷 기반 SQL 분석, 리포트, 다중 포맷 내보내기와 Hugging Face 게시
기대효과 및 활용 분야	• 공공데이터 가공 과정을 누구나 같은 방식으로 반복하고 설명할 수 있음 • 공공데이터 기반 수업·연구, 데이터 분석, AI 학습용 데이터셋 구축과 배포에 활용

(본문) 프로젝트 결과보고서

Ⅰ. 프로젝트 개요

1. 프로젝트 소개

- 공공데이터를 분석 가능한 테이블로 만들고 그 과정을 확인하는 플랫폼 개발
- 사용자는 Studio에서 소스 데이터셋을 찾아 테이블 생성 규칙을 입력
- Builder가 규칙을 검증한 뒤 수집, 단계별 변환, 결과 기록을 수행
- 만들어진 테이블은 스냅샷으로 보관되어 SQL 분석, 리포트, 내보내기에 사용
- 주요 결과물: 실행·웨어하우스 엔진 KPubData Builder(Python), 웹 작업공간 KPubData Studio(React·TypeScript), 실행 계약 BuildSpec·Manifest

2. 개발 배경 및 필요성

- 공공 API 응답을 받은 뒤에도 자료형 정리, 품질 확인, 재실행 기준, 파일 생성 작업이 남음
- 이 과정을 스크립트와 수작업으로 처리하면 어떤 입력과 규칙으로 결과가 나왔는지 알기 어려움
- 실행 입력을 BuildSpec으로 선언하고 Bronze·Silver·Gold로 역할을 나누어 결과를 추적할 수 있게 설계
- 명세 파일과 API만으로는 진입 장벽이 높아, 같은 기능을 화면으로 다룰 수 있도록 Studio 개발

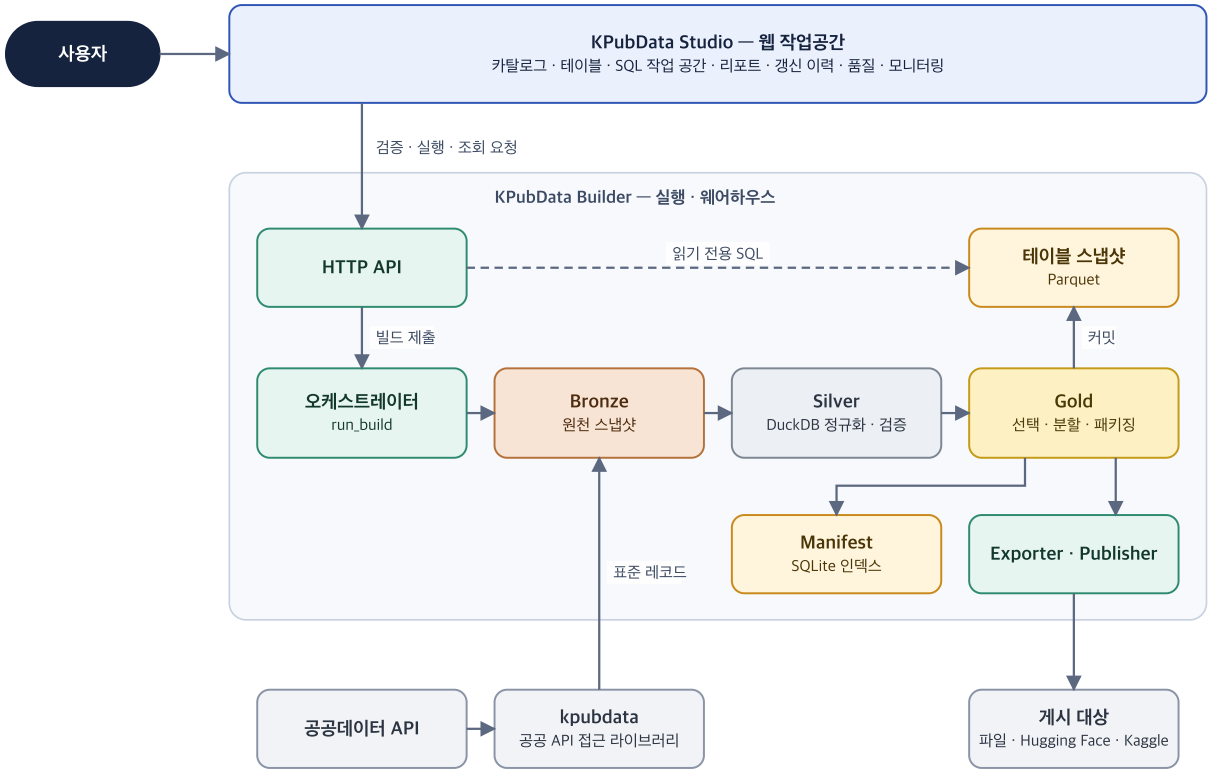
3. 프로젝트 특·장점

구분	기존 방식	본 프로젝트
공공데이터 포털	원천 API 제공까지. 가공은 이용자 몫	정제·검증·분할·문서화를 거친 테이블과 데이터셋 산출
범용 ETL 도구	파이프라인은 실행되나 검증 규칙은 직접 작성	스키마 계약과 품질 기준을 명세에 선언하고 실행 전에 검증
데이터 허브	배포·공유 중심. 생성 과정은 남지 않음	실행 기록과 입력 지문을 Manifest로 남기고 카드와 함께 게시
LLM 데이터 도구	LLM이 내용을 추정해 생성	답변을 실제 근거와 대조하고 사용자 승인 전에는 실행하지 않음

- 책임 분리: Studio는 화면, Builder는 검증·실행·질의, kpubdata는 공공 API 접근 담당
- 안전 장치: 개인정보 검사, 라이선스 확인, 읽기 전용 SQL 검사를 실행 경로에 포함

Ⅱ. 프로젝트 내용

1. 프로젝트 구성도



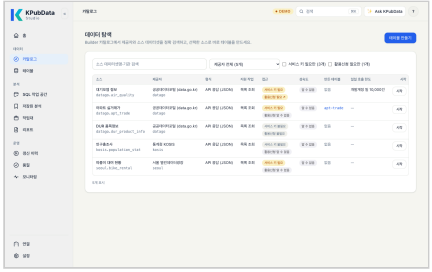
- Studio는 Builder API만 호출하며 실행 엔진을 갖지 않음
- Builder는 kpubdata로 원천을 받아 Bronze → Silver → Gold 순으로 처리
- 성공한 실행의 결과는 테이블 스냅샷으로 커밋되고 실행 내역은 Manifest로 기록

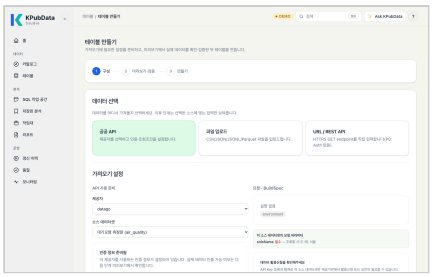
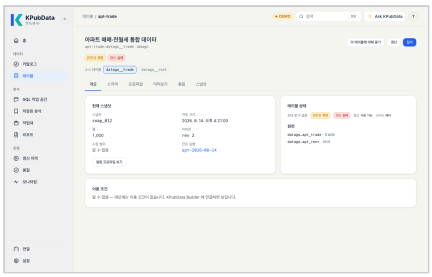
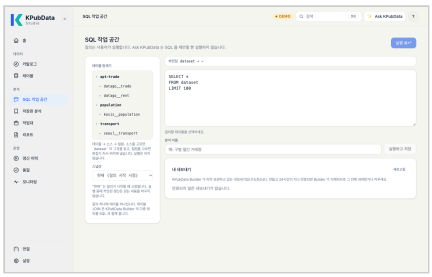
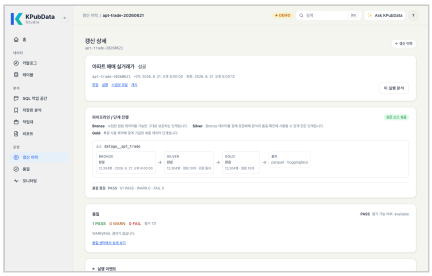
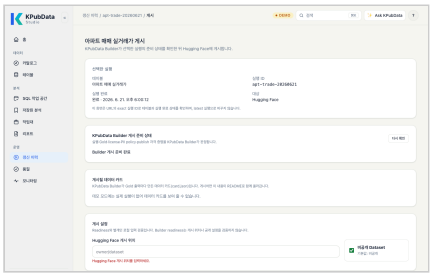
2. 프로젝트 기능

1) 전체 기능 목록

구분	기능	설명
S/W	BuildSpec 검증·미리보기	명세를 실행 전에 검사하고 저장 없이 스키마와 샘플 확인
	단계별 실행	Bronze 수집, Silver 정제·검증, Gold 패키징을 소스별로 병렬 수행
	품질·개인정보 검사	중복률·결측률·값 범위 등 품질 검사와 개인정보 패턴 검사, 마스킹
	실행 기록	상태·출처·행 수·품질 결과를 Manifest와 SQLite 인덱스로 기록
	테이블 스냅샷	성공한 결과를 읽기 전용 스냅샷으로 커밋하고 이력 유지
	내보내기·게시	CSV·JSONL·Parquet·Markdown 등으로 내보내고 Hugging Face·Kaggle에 게시
	카탈로그·테이블 만들기	소스를 찾아 3단계 마법사로 테이블 생성
	테이블·테이블 상세	스냅샷, 스키마, 프로파일, 미리보기, 품질 확인
	SQL 작업 공간	스냅샷에 읽기 전용 SQL 실행, 분석 저장, 차트, 결과 내보내기
	리포트	실행 근거를 바탕으로 리포트 작성, Markdown·HTML 내려받기
	갱신 이력·품질·모니터링	실행 목록과 상세, 품질 경고, Builder 상태 확인
	Ask KPubData	화면 문맥과 근거를 바탕으로 답변·SQL·다음 동작을 제안하는 AI 도우미
	연결·설정	제공 기관 자격 증명 관리, OIDC 로그인, Builder 버전 확인
H/W	해당 없음	전용 하드웨어를 사용하지 않는 소프트웨어 프로젝트

2) S/W 주요 기능

기능	설명	프로젝트실물사진
공공데이터 탐색	• 카탈로그에서 소스 데이터셋 검색 • 제공 기관, 서비스 키 필요 여부로 필터 • 시작 버튼으로 테이블 만들기 진입	

테이블 만들기	• 구성 → 미리보기·검증 → 만들기 3단계 • 공공 API, 파일 업로드, URL 소스 지원 • 검증을 통과해야 실행 가능	
테이블 관리	• 테이블별 현재 스냅샷과 행 수 확인 • 조치가 필요한 상태 표시 • 상세에서 스키마·품질·스냅샷 이력 확인	
SQL 분석	• 스냅샷을 골라 읽기 전용 SQL 실행 • 분석 저장과 재실행 • 집계 차트, 결과 내보내기	
실행 확인	• 소스별 단계 진행과 품질 결과 확인 • 실행 이벤트 타임라인 • 실패 시 단계와 코드 표시, 재실행	
게시	• 게시 준비 상태와 차단 사유 확인 • 데이터 카드 미리보기 • Hugging Face 저장소로 게시	

- 화면은 데모 데이터로 실행한 Studio를 캡처한 것임 실제 운영 화면 확보 시 교체

3) H/W 주요 기능

기능/부품	설명	프로젝트실물사진
해당 없음	-	-

3. 주요 적용 기술

- **선언형 실행 계약 (BuildSpec)**
  - 소스, 스키마 계약, 품질 기준, 분할, 내보내기를 하나의 명세로 선언
  - 실행 전에 15개 영역을 검사해 문제를 한 번에 알려 줌
- **Medallion 단계 처리**
  - Bronze: 원천 레코드와 수집 시각·파라미터·체크섬 보존
  - Silver: 선언한 규칙만 적용해 정규화하고 스키마·품질·개인정보 검사
  - Gold: 컬럼 선택, 마스킹, 분할, 조인 후 패키징
- **DuckDB·Parquet**
  - 표 처리 엔진을 DuckDB 하나로 통일
  - 결과는 Parquet 스냅샷으로 저장하고 격리된 연결에서 읽기 전용으로 질의
- **실행 기록과 스냅샷**
  - 실행마다 Manifest 한 건 기록, SQLite 인덱스로 조회
  - 스냅샷 커밋은 revision 비교로 동시 갱신 충돌 방지
- **읽기 전용 SQL 검사**
  - sqlglot으로 구문을 분석해 단일 SELECT만 허용
  - 테이블 함수, 비결정 함수, 외부 접근 차단
- **AI 응답 검증**
  - LLM 답변을 구조 검증 → 근거 대조 → Builder 검증 → 사용자 승인 순으로 확인
  - 전송 전에 키·토큰 등 비밀값 제거
- **인증**
  - Keycloak 기반 OIDC (Authorization Code + PKCE), 토큰은 메모리에만 보관

4. 프로젝트 개발 환경

구분		상세내용
S/W 개발환경	OS	개발 장비 OS 기재 / 배포: Linux 컨테이너 (Docker)
	개발환경(IDE)	사용 IDE 기재
	개발도구	Builder: DuckDB, pyarrow, SQLite, sqlglot, uv / Studio: React 19, Vite 8, React Router 7, Zustand 5, Tailwind CSS 4, Zod 4
	개발언어	Python 3.10~3.13 (Builder), TypeScript 6 (Studio)
	기타사항	테스트: pytest·ruff·mypy, Vitest·Playwright / 인증: Keycloak
H/W 구성장비	디바이스	해당 없음
	센서	해당 없음
	통신	해당 없음
	언어	해당 없음
	기타사항	해당 없음
프로젝트 관리환경	형상관리	Git, GitHub (kpubdata-lab 조직 저장소)
	의사소통관리	GitHub Issues·Pull Request 메신저·회의 도구 기재
	기타사항	GitHub Actions (CI, 정기 수집, 이미지 빌드)

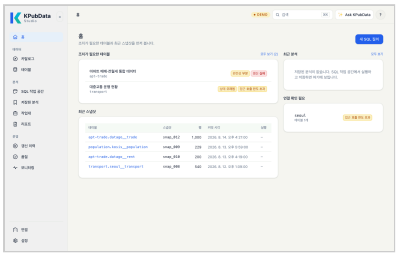
5. 장비(기자재/재료) 활용

번호	품명	작품에서의 주요기능
1	개발용 컴퓨터	Builder·Studio 개발과 시연 기종 기재
2	서버 또는 클라우드	배포 환경이 있으면 기재

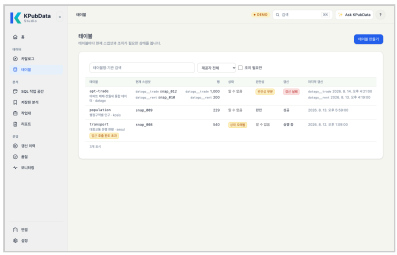
6. 프로젝트 작동 동영상

- 시연 영상 유튜브 링크 기재

7. 결과물 상세 이미지



홈: 조치가 필요한 테이블과 최근 스냅샷



테이블: 스냅샷과 상태



품질: 검사 결과와 스키마 변화

8. 달성 성과

☐ 논문게재 및 포스터발표	게재(발표)자명	논문(포스터)명	게재(발표)처	게재(발표)일자
☐ 앱(APP) 등록	등록자명	앱(APP)명	등록처	등록일자
☐ 프로그램 등록	등록자명	프로그램명	등록처	등록일자
☐ 특허/실용신안 출원	출원자명	특허/실용신안명	출원번호	출원일자
☐ 기술이전	기술이전기업명	기술명	금액	이전일자
☐ 공모전	구분(교내/대외)	공모전명	수상여부(출품/수상)	상격
☐ 실용화				
☐ 기타				

- 해당 성과를 체크하고 내용 기재. 증빙은 별첨1에 첨부

Ⅲ. 프로젝트 수행 내용

1. 업무분장

번호	성명	역할	담당업무
1		멘토	기재
2		지도교수	없으면 공란
3		팀장	기재
4		팀원2	기재
5		팀원3	기재
6		팀원4	기재
7		팀원5	기재

2. 프로젝트 수행일정

구분	추진내용	수행일정								
		3월	4월	5월	6월	7월	8월	9월	10월	11월
계획	주제 선정, 문제 정의, 수행계획서 작성									
분석	공공 API 조사, 요구사항 정의									
설계	BuildSpec·Manifest 계약, 단계별 처리 구조									
	메뉴·화면 설계, 제작설계서 작성									
개발	Builder 파이프라인, 품질·개인정보 검사									
	웨어하우스 스냅샷, SQL 질의, 게시									
	Studio 화면, Ask KPubData									
테스트	단위·통합·화면 테스트, 실연동 점검									
종료	결과보고서 작성, 시연 영상 제작									
온·오프라인 미팅	멘토링·팀 회의 기재									

- 단계별 수행 기간을 노란색 셀로 표시

3. 프로젝트 추진 과정에서의 문제점 및 해결방안

1) 프로젝트 관리 측면

- 일정·협업에서 겪은 문제와 해결 과정 기재

2) 프로젝트 개발 측면

- **대용량 데이터 처리 한계**
  - 문제: 수집부터 질의까지 여러 지점에서 전체 데이터를 메모리에 올려, 약 1.4GB 파일 소스의 빌드가 실패
  - 해결: 표 처리 엔진을 Polars에서 DuckDB로 바꾸고 단계 사이를 Parquet 파일로 연결. 기존 게시 스크립트만 이전 엔진 유지
- **실패한 실행의 상태 표시**
  - 문제: Manifest가 만들어지기 전에 끝난 실행은 기록이 없어 화면에서 사라짐
  - 해결: 종료 기록을 따로 남기고, 실행 결과(ok)와 API 작업 상태(succeeded)를 한 함수에서 판정
- **테이블 동시 갱신**
  - 문제: 같은 테이블을 두 실행이 동시에 갱신하면 나중 결과가 먼저 결과를 덮어씀
  - 해결: 시작 시점의 revision을 기억했다가 커밋할 때 비교해 다르면 거부
- **AI 답변의 신뢰성**
  - 문제: LLM은 존재하지 않는 테이블이나 실행을 근거로 답할 수 있음
  - 해결: 답변의 참조를 Builder 근거와 대조해 없는 항목을 제거하고, SQL과 동작은 사용자 승인 후에만 실행
- **화면 구조의 복잡도**
  - 문제: 초기 메뉴가 Discover·Add Data·Builds 등 실행 엔진의 용어 중심으로 나뉘어 있었음
  - 해결: 카탈로그 → 테이블 → 분석 → 운영 순으로 메뉴를 다시 묶고, 테이블 만들기를 3단계로 줄임
- **화면에 보이는 값의 정확성**
  - 문제: Builder가 주지 않는 값을 0이나 임의 점수로 보여 주면 오해 소지
  - 해결: 품질은 점수 대신 검사 결과만 표시하고, 받지 못한 값은 '확인 불가'로 표기

4. 프로젝트를 통해 배우거나 느낀 점

- 팀원별 소감 기재

Ⅳ. 기대효과 및 활용분야

1. 프로젝트의 기대효과

- 공공데이터 가공 과정을 명세와 실행 기록으로 남겨 누구나 같은 결과를 다시 만들 수 있음
- 기관마다 다른 형식을 표준 테이블로 맞추는 반복 작업을 줄임
- 품질·개인정보·라이선스 확인이 실행 과정에 포함되어 배포 전 점검 누락을 방지
- 코드를 작성하지 않는 사용자도 화면에서 테이블 생성과 SQL 분석 가능
- 공개 저장소로 제공되어 누구나 내려받아 사용하고 확장할 수 있음

2. 프로젝트의 활용분야

- 교육·연구: 공공데이터 기반 수업 자료와 연구용 데이터셋 구축
- 데이터 분석: 주기적으로 갱신되는 테이블 구축과 SQL 분석, 리포트 작성
- AI 개발: 출처와 처리 내역이 기록된 학습용 데이터셋 구축과 허브 배포
- 공공·민간 서비스: 공공데이터를 활용하는 서비스의 데이터 준비 단계

Ⅴ. 참고자료

1. 참고 및 인용자료

- KPubData Builder 저장소: github.com/kpubdata-lab/kpubdata-builder
- KPubData Studio 저장소: github.com/kpubdata-lab/kpubdata-studio
- kpubdata 저장소: github.com/kpubdata-lab/kpubdata
- 공공데이터포털: www.data.go.kr
- DuckDB 문서: duckdb.org/docs
- SW개발/HW제작 설계서 (26_HC277), 2026. 10. 12 개정

별첨1 2026년 한이음 드림업 프로젝트 성과 증빙자료

- 프로그램 등록 증명서, 논문·학술대회 초록집 등 성과 증빙 이미지 첨부